

Lesson 3 Color Sorting

1. Program Logic

First, program ArmPi to recognize colors with Lab color space. Convert the RGB color space to Lab, image binarization, and then perform operations such as expansion and corrosion to obtain an outline containing only the target color. Use circles to frame the color outline to realize object color recognition.

Secondly, judge whether the object moves after recognition. Obtain the center coordinates of the object, transform the coordinates into reality, and then compare the last coordinates by drawing the center point to determine whether it is a movement. If the object is still moving, the robotic arm will track it. When the object stops moving for few seconds, the robotic arm will grip it.

Secondly, program to decide the order of gripping. There are 2 situations for the gripping sequence.

Case One: Place red, green and blue block in the recognition area in advance.

ArmPi will grip in the order of red, green and blue (prioritize the color that is recognized first).

Case Two: Place three same color blocks in the area in advance such as 3 red blocks. By calculating and comparing the recognition area of the objects, the object with largest area is preferred.

Last step is to sort the gripped blocks and then back to the initial position.

The source code of the program is located in :
</home/pi/ArmPi/Functions/ColorSorting.py>

```

if not start_pick_up:
    if color_area_max == 'red': # red area is the maximum
        color = 1
    elif color_area_max == 'green': # green area is the maximum
        color = 2
    elif color_area_max == 'blue': # blue area is the maximum
        color = 3
    else:
        color = 0
    color_list.append(color)
    # cumulative judgment
    if distance < 0.5:
        count += 1
        center_list.extend((world_x, world_y))
        if start_count_tl:
            start_count_tl = False
            tl = time.time()
        if time.time() - tl > 0.5:
            rotation_angle = rect[2]
            start_count_tl = True
            world_X, world_Y = np.mean(np.array(center_list).reshape(count, 2), axis=0)
            center_list = []
            count = 0
            start_pick_up = True
    else:
        tl = time.time()
        start_count_tl = True
        center_list = []
        count = 0

if len(color_list) == 3: # multiple judgments
    # take evaluation value
    color = int(round(np.mean(np.array(color_list))))
    color_list = []
    if color == 1:
        detect_color = 'red'
        draw_color = range_rgb["red"]
    elif color == 2:
        detect_color = 'green'
        draw_color = range_rgb["green"]
    elif color == 3:
        detect_color = 'blue'
        draw_color = range_rgb["blue"]
    else:
        detect_color = 'None'
        draw_color = range_rgb["black"]

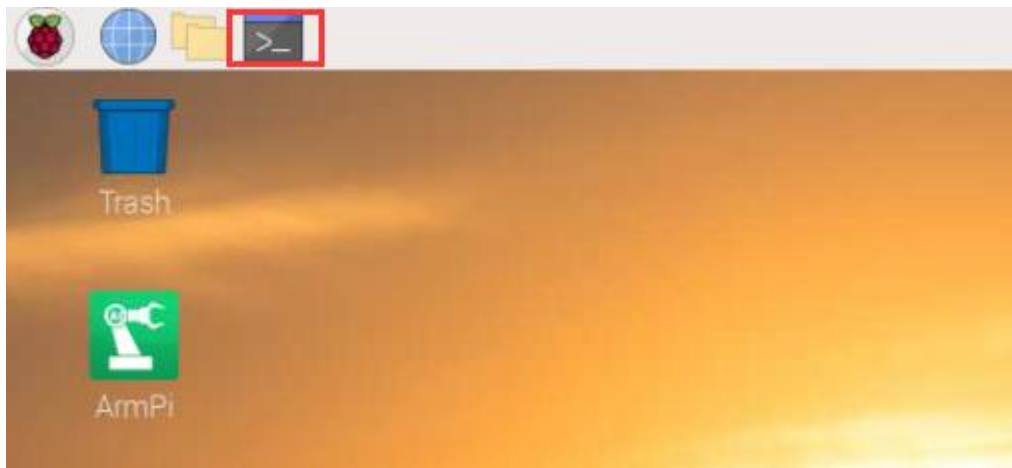
```

2. Operation Steps

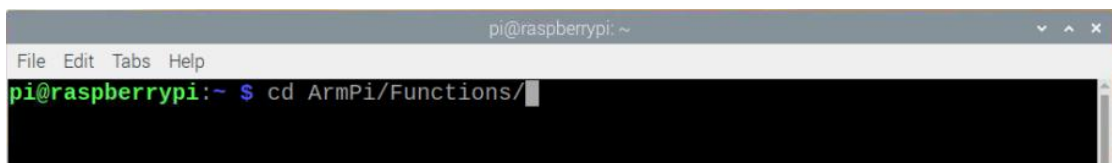
i Pay attention to the text format in the input of instructions.

1) Turn on robotic arm and connect to Raspberry Pi desktop with VNC.

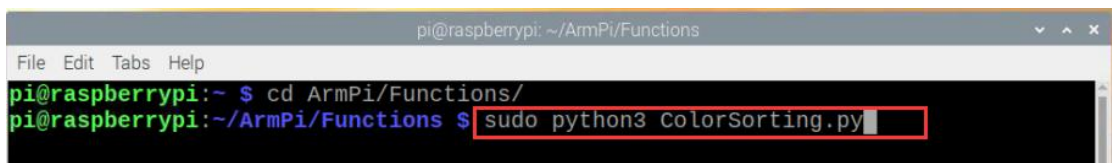
2) Click  or press “Ctrl+Alt+T” to open the LX terminal.



3) Enter “cd ArmPi/Functions/” command, and then press “Enter” to come to the category of games programmings.



4) Enter “sudo python3 ColorSorting.py”, then press “Enter” to start the game.



5) If you want to exit the game programming, press “Ctrl+C” in the LX terminal interface. If the exit fails, you can press it multiple times.

3. Project Outcome

i In order to grip easily, please do not place the blocks too close.

Place three color blocks separately in the camera recognition area. When ArmPi recognizes the blue block, the buzzer on the Raspberry Pi expansion board will beep, and the RGB lights will light in blue. ArmPi will put the blue block in B area and then continue to sort other objects.

